

Full Source Code – RAG with LlamaIndex

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

Chat_example.py

Found in Project root folder

This code loads a data set of “documents” using the SimpleDirectoryReader class from the llama_index module. It then creates an index and transforms it into a chat engine. Finally, it runs a chat bot in reactive mode that gives verbose output.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents

documents = SimpleDirectoryReader('documents').load_data()

# create index

index = VectorStoreIndex.from_documents(documents)

# create chatbot

chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')

chat_engine.chat_repl()
```

Save_index.py

Found in Project root folder

This code imports necessary functions, reads and loads data from a specified directory, constructs an index from the loaded documents, and then saves that index. It’s designed to create an organized structure(Index) for faster search and retrieval of data from the documents directory.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents

documents = SimpleDirectoryReader('documents').load_data()

# create index
```

```
index = VectorStoreIndex.from_documents(documents)
```

```
# save index
```

```
index.storage_context.persist()
```

Query_index.py

Found in Project root folder

This code is responsible for initializing a StorageContext from a pre-defined directory, loading an index from storage, and setting up a query engine. It then queries for a specific piece of information and prints the response.

```
from llama_index.core import StorageContext, load_index_from_storage
```

```
# rebuilding storage context
```

```
storage_context = StorageContext.from_defaults(persist_dir='./storage')
```

```
# load index
```

```
index = load_index_from_storage(storage_context)
```

```
query_engine = index.as_query_engine(verbose=True)
```

```
response = query_engine.query('give me the link to the article with instructions on how to  
create a new teacher account')
```

```
print(response)
```

Custom_llm.py

Found in Project root folder

This code sets up a chatbot that uses the OpenAI GPT-4 model for natural language understanding, using the Llama-based indexing system. It reads in a set of documents, uses them to create a knowledge index, and then allows for interactive, real-time chat based on this index.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
```

```
from llama_index.llms.openai import OpenAI
```

```
import openai
```

```
openai.log = 'debug'
```

```
# define LLM
```

```

llm = OpenAI(model="gpt-4", temperature=0, max_tokens=1000)

# configure service context

Settings.llm = llm

# load documents

documents = SimpleDirectoryReader('documents').load_data()

# create index

index = VectorStoreIndex.from_documents(documents)

# create chatbot

chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')

chat_engine.chat_repl()

```

Metadata.py

Found in Project root folder

This code imports classes from a llama_index module, loads and categorizes two types of documents for customer support and marketing respectively. It generates a combined index from these categorized documents and saves it. Finally, a chat engine is created using this index and a chat session is started.

```

from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents

docs1 = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge
base.csv']).load_data()

docs2 = SimpleDirectoryReader('docs2').load_data()

docs1[0].metadata['category'] = 'customer support'

for doc in docs2:

    doc.metadata['category'] = 'marketing'

all_docs = docs1 + docs2

# create index

index = VectorStoreIndex.from_documents(all_docs)

```

```
# save index

index.storage_context.persist()

# create chatbot

chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')

chat_engine.chat_repl()
```

Summarization_starter.py

Found in Project root folder

This code loads documents from a specified file, assigns a category to them, and creates an index. The index is then saved before a query is made to summarize content related to customer support. Finally, the query response is printed out.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents

docs = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge
base.csv']).load_data()

docs[0].metadata['category'] = 'customer support'

# create index

index = VectorStoreIndex.from_documents(docs)

# save index

index.storage_context.persist()

# query

query_engine = index.as_query_engine(verbose=True)

response = query_engine.query('summarize our customer support content')

print(response)
```

Summarization.py

Found in Project root folder

This code reads a set of documents from a directory and categorizes them as ‘customer support’. It then creates an index for the documents and stores it. The last part of the code

queries the indexed documents for customer support content and outputs a summary response, additionally printing relevant sources.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, SummaryIndex

# load documents

docs = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge
base.csv']).load_data()

docs[0].metadata['category'] = 'customer support'

# create index

index = VectorStoreIndex.from_documents(docs)

# save index

index.storage_context.persist()

# query

query_engine = index.as_query_engine(verbose=True, similarity_top_k = 6,
response_mode='tree_summarize')

response = query_engine.query('summarize our customer support content')

print(response)

print(response.get_formatted_sources())

#print(response.source_nodes)
```